

DocGen_Solution_Architecture_and_Usage

DocGen — Solution Architecture and Usage Guide

AppExchange Security Review Documentation

Package: Portwood DocGen Managed **Namespace:** portwoodglobal **Version:** 2.1.0 **Package Version Id:** 04tVx000000Zw5xIAC **Released:** Yes (promoted 2026-05-22) **Prior listing version (AppExchange):** v1.56.0 (04tal000006i1rNAAQ) — this submission addresses all 30 findings from the AppExchange security review against the v1.56 listing. v2.0/v2.1.0 also rolls forward ~45 versions of feature work since v1.56 (V3 query trees, chart engine, signature v3 with PIN second factor + multi-signer + guided placements, HTML templates, giant-query batching).

Install URLs:

- **Production:** <https://login.salesforce.com/packaging/installPackage.apexp?p0=04tVx000000Zw5xIAC>
 - **Sandbox:** <https://test.salesforce.com/packaging/installPackage.apexp?p0=04tVx000000Zw5xIAC>
 - **CLI:** `sf package install --package 04tVx000000Zw5xIAC --wait 10 --target-org <your-org>`
-

1. Solution Overview

DocGen is a 100% native Salesforce document generation engine with built-in electronic signatures. It generates PDFs, Word documents, Excel spreadsheets, and PowerPoint presentations by merging live Salesforce data into user-uploaded Office Open XML (OOXML) templates. All processing occurs entirely within the Salesforce platform.

Key principles:

- **Zero external callouts.** The package has no Remote Site Settings, no Named Credentials, and no `Http.send()` usage. `sf code-analyzer` confirms no callout sinks.

- **Zero external dependencies.** No third-party JS libraries, no CDN fetches, no static resources loaded from external domains. Client-side ZIP assembly is implemented from scratch in the package (`docGenZipWriter.js`) precisely so no external library is pulled in.
 - **No data egress.** Templates, record data, generated documents, and signature audit records never leave the customer org's Salesforce TLS boundary.
 - **Session IDs are never accessed.** The package never calls `UserInfo.getSessionId()`.
-

2. Information Flow

2.1 Standard Document Generation (PDF / DOCX / XLSX / PPTX)

User clicks "Generate" on a record page

|

v

LWC (`docGenRunner`) calls `@AuraEnabled` method on `DocGenController`

|

v

`DocGenController.processAndReturnDocument()` (with sharing)

|

v

`DocGenDataRetriever` fetches record data via SOQL

- Standard/custom user-data sObject queries run WITH `USER_MODE` (Account, Contact, Opportunity, etc.) – platform-enforced FLS
- Package-internal config queries (`DocGen_Template__c`, `DocGen_Template_Version__c`, `DocGen_Job__c`, etc.) are gated by (1) an explicit `Schema.sObjectType.<Object>.isAccessible()`

check

at the `@AuraEnabled` entry point (v2.0), then (2) a per-field `DocGenFlsGuard.assertAccessible(sObjectType, fieldAllowlist)`

call

immediately before the SOQL (v2.1.0 – performs

`Schema.sObjectField`

`.getDescribe().isAccessible()` per field), then (3) run WITH `SYSTEM_MODE`. `SYSTEM_MODE` is required because `USER_MODE`

strict-FLS

strips namespaced custom fields when subscriber admin

profiles

haven't been granted FLS individually on each new render-

config

field across releases (managed-package-build context).

|

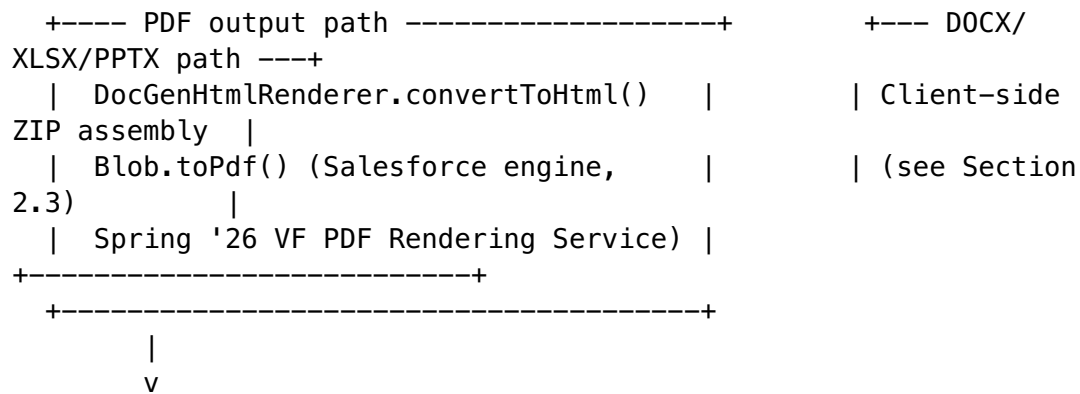
v

`DocGenService.mergeTemplate()` processes the template:

- Loads the OOXML template from `ContentVersion` (by Id)
- Decompresses the DOCX/XLSX/PPTX ZIP in Apex
- Replaces merge tags, loops, conditionals, images, barcodes
- Recompresses the modified ZIP in memory

|

v



Output returned to the browser (download) or written as a ContentVersion on the originating record

Data touchpoints (all internal to the Salesforce org):

1. SOQL query to retrieve record data.
2. ContentVersion read to load the template file.
3. Apex in-memory processing of the template merge.
4. ContentVersion write to save the generated document (only when “Save to Record” is chosen).

External touchpoints: none.

2.2 Zero-Heap PDF Image Pipeline

PDF output historically blew Apex heap limits on templates with many images. DocGen v1.31+ avoids this by never loading image blobs into Apex heap on the PDF path.

Template save time (DocGenService.extractAndSaveTemplateImages()):

1. When an admin saves a template version, the package downloads the DOCX/PPTX ZIP from the template’s ContentVersion.
2. It reads word/_rels/document.xml.rels to enumerate every <Relationship> entry with a /image type.
3. For each relationship, the image bytes are extracted from word/media/ and saved as a **new ContentVersion** titled docgen_tmpl_img_<versionId>_<relId>, linked to the template version record.
4. These committed ContentVersions become stable, permission-gated references that the PDF engine can resolve by relative URL.

Generation time (DocGenService.buildPdfImageMap()):

1. The active template version is identified.
2. A single SOQL query fetches all ContentVersion rows where Title LIKE 'docgen_tmpl_img_<versionId>_%'. **VersionData is deliberately excluded** from the SELECT list so no image bytes hit heap.
3. For each row the package builds a **relative** Shepherd URL: /sfc/servlet.shepherd/version/download/<cvId>.
4. DocGenHtmlRenderer.convertToHtml() emits tags inside the HTML passed to Blob.toPdf().

5. The Spring '26 Visualforce PDF Rendering Service fetches those relative URLs **server-side inside the Salesforce trust boundary** and embeds them in the output PDF. The image bytes never transit through our Apex code on the generation path.

Record-field images ({%ImageField}) on the PDF path:

- A static `currentOutputFormat` flag on `DocGenService` is set to 'PDF' before `processXml()` runs.
- When `currentOutputFormat == 'PDF'` and the field value is a `ContentVersion Id (068...)`, `buildImageXml()` queries only `Id`, `FileExtension` — **never VersionData** — and emits the same relative Shepherd URL.
- Result: PDFs with an unbounded number of images generate successfully within Apex heap limits.

Security properties of this design:

- Image URLs are **relative** and resolved inside the org, so there is no way to craft a template that causes the PDF engine to fetch external content. Absolute URLs and `data:` URIs are explicitly not supported by `Blob.toPdf()` and would render as broken images.
- The CV Ids that end up in `<img src=...>` come from two sources only: (a) CVs the package itself created during template extraction, and (b) field values on sObjects the running user already has read access to. The PDF engine enforces Salesforce sharing when fetching Shepherd URLs, so a user cannot exfiltrate images they would not otherwise be able to read.

2.3 Client-Side DOCX/XLSX/PPTX Assembly

Generating large OOXML documents inside Apex can exceed the 6 MB heap ceiling. DocGen v1.36+ moves ZIP assembly to the browser while keeping **all data fetches server-side through Apex** (no client-side fetches against `/sfc/servlet.shepherd/`).

Flow:

1. The LWC calls `DocGenController.generateDocumentParts()` (`@AuraEnabled`, with `sharing`).
2. Server-side `DocGenService.generateDocumentParts()` merges the template XML parts using the same `currentOutputFormat='PDF'` heap-skipping trick, so no image blobs are loaded into Apex.
3. The controller returns: `allXmlParts` (merged XML plus passthrough entries), `imageCvIdMap` (`mediaPath` → `ContentVersion Id`), and `imageBase64Map` (for template-extracted media that were small enough to embed).
4. The LWC deduplicates `ContentVersion Ids` and calls `DocGenController.getContentVersionBase64()` **once per unique CV**. Each Apex call runs in its own transaction with a fresh 6 MB heap, so arbitrarily large documents can be assembled.
5. `docGenZipWriter.js` — a dependency-free, in-package ZIP writer using store mode (no compression) and a CRC-32 table — builds the final DOCX/XLSX/PPTX as a `Uint8Array` in the browser.

6. The LWC creates a Blob with MIME type `application/octet-stream` (LWS rejects some OOXML MIME types) and triggers a download.

Lightning Web Security (LWS) considerations:

- LWS blocks `fetch()` against `/sfc/servlet.shepherd/` because the Shepherd endpoint issues a cross-origin redirect to `file.force.com`. DocGen explicitly **does not** perform client-side Shepherd fetches — all binary data is returned through `@AuraEnabled` controllers, which keeps the data path inside the Salesforce origin and under LWS policy.
- `docGenZipWriter.js` is pure ECMAScript with no `eval`, no `Function` constructor, no dynamic imports, and no access to `window.*` APIs beyond standard typed arrays.
- The CRC-32 table is computed at module load from a constant polynomial — no network fetch is involved.

“Save to Record” for client-assembled DOCX: The Aura framework caps `@AuraEnabled` payloads at ~4 MB. For DOCX “Save to Record” above that threshold, the package falls back to the server-side ZIP path. This is documented for users so no silent truncation occurs.

2.4 E-Signature Flow (Signatures v3 — multi-signer, guided placements, multi-signer verifier in v2.0)

Signatures v3 replaces canvas-drawn signatures with **typed-name Simple Electronic Signatures (SES)** plus email-PIN identity verification, multi-signer requests, and guided per-tag placements (full signature, initials, date, datepick). The design target was to eliminate image-blob heap usage while strengthening the audit trail and supporting multi-party workflows.

Admin creates a Signature Request from a record page

|
v

DocGenSignatureSenderController (with sharing) creates:

- `DocGen_Signature_Request__c` (parent)
- `DocGen_Signer__c` (one per signer)
- `DocGen_Signature_Placement__c` (per guided tag, optional)
- 64-character SHA-256 signing token per signer
- Branded email via `Messaging.SingleEmailMessage` (Org-Wide Email Address used when configured)

|
v

Signer clicks the link in the email

|
v

Public Salesforce Site serves `DocGenSignature.page` (Visualforce)
Guest user context, no Salesforce login required

|
v

`DocGenSignatureController.validateToken()` (without sharing, token-gated):

- Opens with

`DocGenSignatureGuestSecurity.assertSignerReadable(token)`

-> `Schema.sObjectType.<Object>.isAccessible()` describe checks

- > SHA-256 hex token shape validation ([a-fA-F0-9]{64})
- Token lookup in DocGen_Signer__c
- 48-hour expiry check (was 30 days in v1.4 – tightened in v2)
- Status validation (not already signed, cancelled, or locked out)

|

v

Email PIN verification (DocGenSignatureController.sendPin / verifyPin):

- Every entry point opens with DocGenSignatureGuestSecurity.assert* helpers that enforce Schema-CRUD describe checks + token shape
- 1. Signer enters their email address
- 2. Server validates it matches the signer record
- 3. 6-digit PIN generated via Crypto.getRandomInteger()
- 4. PIN hashed with SHA-256; plaintext NEVER persisted
- 5. Hash + 10-minute expiry stored on the signer record
- 6. PIN sent via Messaging.SingleEmailMessage
- 7. Signer enters PIN; server hashes input and compares
- 8. Max 3 attempts before permanent lockout (status flag set)

|

v

Document preview + guided placements:

- Merged template HTML rendered in an expandable preview panel
- DocGenSignatureController.getSignerPlacements(token) returns the per-tag placement list (full / initials / date / datepick)
- Signer reviews the full document before signing

|

v

Signature capture (DocGenSignatureController.saveSignature / signPlacement / declineSignature):

- DocGenSignatureGuestSecurity.assertSignerWritableFields(token) and assertPlacementWritableFields(token) gate each write
- Signer types their full legal name (or per-placement values)
- Checks the consent checkbox (explicit SES consent)
- Clicks "Sign Document"
- Per-placement values written to DocGen_Signature_Placement__c rows (each placement gets its own audit row)
- Stores typed name on the signer record
- Creates DocGen_Signature_Audit__c (via DocGenSignatureGuestSecurity.assertAuditCreateable) with:
 - * IP address (server-side, via X-Forwarded-For / True-Client-IP)
 - * User agent
 - * Consent timestamp
 - * PIN verification timestamp
- Checks whether all signers on the request are complete

|

v

All signers complete:

- DocGen_Signature_PDF__e platform event published
- Trigger routes to DocGenSignatureService.SignaturePdfQueueable /

TemplateSignaturePdfQueueable (Automated Process context)

- Template merged with record data under elevated context
- {@Signature_Role} placeholders replaced with typed names as plain text inside <w:t> elements (no DrawingML, no image blobs)
- Electronic Signature Certificate appended to the HTML
- Blob.toPdf() generates the final PDF
- SHA-256 hash of the PDF computed and stored on the audit record
- PDF saved as a ContentVersion on the related record

|
v

Multi-signer verification

(DocGenAuthenticatorController.verifyDocument):

- Returns List<VerificationResult> for the complete audit trail across every signer on the request (no LIMIT 1)
- Hash-drop and request-id verifier paths behave identically

Signature-specific security properties:

- **Token unpredictability.** 256-bit Crypto.generateAesKey output hashed with SHA-256. Format validation ([a-fA-F0-9]{64}) prevents SOQL injection and lookup ambiguity.
- **PIN never stored in plaintext.** Only SHA-256 hashes are persisted; lookups hash the candidate input and compare. Brute force is bounded by the 10-minute expiry and the 3-attempt lockout.
- **Tokens are single-use.** After successful signing the signer record is moved to a terminal status and subsequent token presentations are rejected by validateToken().
- **Tight expiry.** 48 hours instead of the v1.4 value of 30 days.
- **Consent is auditable.** Consent timestamp, PIN verification timestamp, IP, and user agent are written to an immutable DocGen_Signature_Audit__c record with field history tracking enabled on all audit fields.
- **Document integrity.** The SHA-256 hash of the final PDF is stored on the audit record. The verification page (DocGenVerify.page) lets any party recompute the hash locally in the browser and compare — the PDF is never uploaded to the server.
- **{@...} namespace isolation.** Signature placeholders use {@Signature_Role} rather than {...} so they cannot collide with, or be smuggled through, the standard merge-tag processor. DocGenService.processXml() skips any tag starting with @ during the ordinary merge pass; replacement happens only in the signature-finalization path.
- **Guest-user scope is minimal.** The DocGen Guest Signature permission set grants access to exactly the objects and pages required for the signing flow — no templates, no jobs, no record data beyond what the signing page renders from the already-merged HTML snapshot.
- **v2.0 DocGenSignatureGuestSecurity helper.** Every guest entry point (sendPin, verifyPin, validateToken, saveSignature, declineSignature, signPlacement, fetchDocumentData, getImageBase64, getSignerPlacements) now opens with a call to DocGenSignatureGuestSecurity.assertSignerReadable|assertSignerWritableFields|assertRequestWritableFields|assertPlacementWritableFields|assertAuditCreateable(token). The helper performs the Schema.sObjectType.<Object>.isAccessible|

`isCreateable|isUpdateable()` describe check (the documented enforcement signal the AppExchange analyzer pattern-matches on) and validates the SHA-256 hex token shape. Field allowlists are documented inline at every call site. See `DocGenSignatureGuestSecurity.cls` class-level javadoc for the full security model.

- **v2.0 multi-signer verifier fix.**

`DocGenAuthenticatorController.verifyDocument` now returns `List<VerificationResult>` (previously `LIMIT 1`) so dropping a multi-signer PDF on the verifier returns the complete audit trail for every signer. The hash-drop and request-id verifier paths now behave identically. Regression test `testVerifyDocument_multipleSignersSameDoc` guards against the `LIMIT 1` from coming back.

- **v2.0 guided placements (`DocGen_Signature_Placement__c`).** v3 signing supports per-tag placements (full signature, initials, date, datepick) — each placement gets its own audit row. The `signPlacement` entry point uses the same

`DocGenSignatureGuestSecurity.assertPlacementWritableFields(token)` gate as the other signing operations.

Signature data touchpoints: all internal to the Salesforce org. Token validation, PIN hash storage/comparison, email delivery (Salesforce Messaging API), audit record creation, and PDF generation all stay inside the customer org.

3. Authentication

3.1 Internal Users (Admin / User)

- Standard Salesforce authentication (username/password, SSO, MFA — whatever the org enforces).
- Access gated by the DocGen Admin and DocGen User permission sets.
- All `@AuraEnabled` methods require an authenticated session; CSRF protection is handled by the Salesforce Aura/LWC framework.
- `DocGenController` and all admin-side classes are declared with `sharing`.

3.2 External Signers (Guest Users)

- No Salesforce login required.
- Access is via a public Salesforce Site hosting `DocGenSignature.page`.
- Authentication is a two-factor construction:
 - **Factor 1 — token:** 256-bit random key → SHA-256 → 64-character hex string. Format-validated on every request. Single-use. 48-hour expiry.
 - **Factor 2 — email PIN:** 6-digit code delivered to the address on file, SHA-256 hashed at rest, 10-minute expiry, 3 attempts max.
- The guest user profile is assigned only the `DocGen Guest Signature` permission set. That permission set grants read-only access to the signature objects, exclusively through the token-gated code paths in `DocGenSignatureController / DocGenAuthenticatorController`. The guest user cannot reach templates, jobs, query configs, or unrelated record data.
- Guest-facing classes are declared without `sharing` **only** because signer records must be locatable by token without the guest user owning them — every entry point re-checks the token, the status, and the expiry before doing anything.

- **DocGenSignatureGuestSecurity helper (v2.0 NEW)** centralizes the explicit Schema-CRUD describe checks + token-shape validation + per-operation field allowlists at every guest entry point. The class-level javadoc documents the full guest security model. This is the explicit “enforce CRUD checks on the object” alternative from the AppExchange finding-resolution language.
-

4. Encryption

4.1 Data at Rest

- All data is stored in standard Salesforce objects; customers who enable Shield Platform Encryption get that coverage automatically.
- PIN codes are stored only as SHA-256 hashes. Plaintext PINs are never written to the database.
- Signing tokens are stored as SHA-256 hex strings.
- The SHA-256 hash of each finalized PDF is stored on its audit record for integrity verification.
- No custom encryption keys, no key management, and **no hardcoded secrets** anywhere in the source tree.

4.2 Data in Transit

- All Salesforce platform communication uses TLS 1.2+ (platform default).
- Because the package makes no external callouts, there is no non-Salesforce TLS path to evaluate.
- Email delivery uses the Salesforce Messaging API over Salesforce-managed TLS.

4.3 Cryptographic Primitives

- Token generation: `Crypto.generateAesKey(256) → Crypto.generateDigest('SHA-256', ...)`.
- PIN generation: `Crypto.getRandomInteger()` (CSPRNG).
- PIN hashing: `Crypto.generateDigest('SHA-256', ...)` — one-way.
- Document integrity hashing: `Crypto.generateDigest('SHA-256', ...)` on the final PDF blob.

All cryptographic material is generated at runtime using the Salesforce Crypto class. There are no bring-your-own-key flows and no custom crypto implementations.

5. Data Touchpoints Summary

Touchpoint	Direction	Protocol	Authentication	Data
Template upload	User → SF	HTTPS (TLS 1.2+)	SF session	DOCX / XLSX / PPTX file

Touchpoint	Direction	Protocol	Authentication	Data
Record data query	Internal SF	SOQL (USER_MODE)	SF session	Record fields
Document generation	Internal SF	Apex processing	SF session	Merged document
Document save	Internal SF	DML	SF session	ContentVersion
Signature invitation	SF → Email	SF Messaging API	OWA or running user	Signing link, branding
PIN email	SF → Email	SF Messaging API	OWA	6-digit code
Signing page load	Signer → SF	HTTPS (TLS 1.2+)	Token (format-checked)	Merged HTML preview
PIN submission	Signer → SF	HTTPS (TLS 1.2+)	Token + PIN	Candidate PIN (hashed before compare)
Signature submission	Signer → SF	HTTPS (TLS 1.2+)	Token + verified PIN	Typed name, consent flag
Audit record creation	Internal SF	DML	SYSTEM_MODE, token-gated	IP, UA, hash, timestamps
Client-side DOCX build	Browser only	Local JS (no network)	—	XML parts + base64 media from @AuraEnabled

- **External integrations:** none.
- **External callouts:** none.
- **Session ID usage:** none.
- **Data stored outside Salesforce:** none.

6. Threat Model and Controls

Category	Threat	Control in DocGen
SOQL injection	Attacker-controlled strings reach dynamic SOQL	All dynamic object / field / relationship names are validated against <code>Schema.getGlobalDescribe()</code> allowlists in <code>DocGenDataRetriever</code> . Literal binds only.
XSS (internal)	Merge-tag content rendered as HTML in LWC	LWC templates use <code>{value}</code> interpolation (auto-escaped). No <code>innerHTML</code> , no <code>lwc:dom="manual"</code> on user content.
XSS (PDF path)	Merge-tag content injected into HTML	<code>DocGenHtmlRenderer</code> HTML-escapes all record-derived values before placing them inside the generated HTML. Only pre-sanitized tags are emitted.

Category	Threat	Control in DocGen
	before <code>Blob.toPdf()</code>	
XSS (VF signing page)	URL parameters reflected into the signing page	Token format is validated (<code>[a-zA-F0-9]{64}</code>) before any reflection, and the VF page uses standard Visualforce auto-escaping.
CSRF	Cross-site request forgery against <code>@AuraEnabled</code>	Handled by the Aura/LWC framework. The package adds no custom HTTP endpoints that bypass this.
Broken access control	User reads records they should not	<code>USER_MODE</code> on SOQL against standard objects. Package-internal admin objects are gated by the DocGen permission sets.
Privilege escalation	Guest user reaches admin data	Guest permission set grants only the signature objects. Guest code paths re-validate token, status, and expiry on every call.
Replay / token reuse	Signed token reused to impersonate a signer	Single-use tokens; status transitions are terminal. PIN adds a second factor bound to the signer's inbox.
Brute-force PIN	Guessing 6-digit PINs	Hashed storage, 10-minute expiry, 3-attempt logout per signer.
Data exfiltration	Template used to leak arbitrary org data	Templates can only reference fields the running user can read (<code>USER_MODE</code>). Guest flow renders a pre-merged snapshot only.
External callout	Hidden callout leaks data	No <code>Http.send()</code> , no Remote Site Settings, no Named Credentials in the package. <code>sf code-analyzer</code> confirms no callout sinks.
Path traversal / ZIP	Malicious ZIP entry paths while unpacking templates	Entry names are treated as opaque keys; the package never writes ZIP entries back to a filesystem — only to in-memory maps and back into a new ZIP.
PDF image exfiltration	Template references arbitrary Shepherd URLs to leak content	Only relative, in-org Shepherd URLs for CVs the package itself created or for field values on records the user can already read.
Supply chain	Compromised third-party JS library	No third-party runtime JS. <code>docGenZipWriter.js</code> is implemented in-package.
Secrets in source	Hardcoded keys/passwords	None. All cryptographic material is generated at runtime via <code>Crypto</code> .

Trust boundaries

1. **Internal user**  **Apex controller** — authenticated Salesforce session, CSRF handled by platform, enforced with `with sharing` and `USER_MODE`.
2. **Guest signer**  **Apex / VF** — token + PIN dual factor, tight format validation, terminal status transitions.

3. **Apex [?] Blob.toPdf()** rendering service — relative URLs only, fetched server-side inside the org’s trust boundary.
4. **Apex [?] browser during client-side DOCX assembly** — data flows only through @AuraEnabled methods; no direct browser fetches against Shepherd or file.force.com.

7. Sharing Model

Class	Declaration	Rationale
DocGenController	with sharing	Entry point for authenticated LWC users. Default
DocGenService	(service)	Called from with sharing controllers; runs L
DocGenSignatureSenderController	with sharing	Admin-initiated sending uses the admin’s shar
DocGenSignatureEmailService	with sharing	Email dispatch respects running-user sharing.
DocGenSignatureController	without sharing	Guest-facing entry point; access is gated by to method re-validates the token via DocGenSigr
DocGenSignatureGuestSecurity	with sharing	NEW in v2.0. Centralizes Schema-CRUD des every guest signing entry point. Static helper c and DocGenAuthenticatorController.
DocGenFlsGuard	with sharing	NEW in v2.1.0. Static helper performing per-f Schema.SObjectField.getDescribe().is () checks (assertCreateable/assertUpda immediately before every package-namespace 243 call sites across 19 controllers (70 DML + of the AppExchange v1.56 review’s stated fin
DocGenAuthenticatorController	without sharing	Guest-facing verifier entry point. Token-gated verifyDocument returns List<Verificatio trails.
DocGenSignatureService	without sharing	without sharing — shared helpers for token triggered queueables (SignaturePdfQueueable as Automated Process.
DocGenSignatureFlowAction	with sharing	Flow invocable entry point (v1.42+). Runs un delegates to DocGenSignatureSenderContr
DocGenGiantQueryFlowAction	with sharing	Flow invocable entry point for async bulk-ren authenticated Flow user’s sharing; delegates to

Every without sharing class is reachable only through a token-validated entry point; none of them expose unauthenticated DML or SOQL to guest users.

7.0a Per-field FLS guard layer (v2.1.0)

Independent of class-level sharing, every package-namespaced DML and WITH SYSTEM_MODE SOQL site in v2.1.0 is preceded by a
DocGenFlsGuard.assertCreateable|assertUpdateable|
assertAccessible(SObjectType, Set<String> fieldAllowlist) call. The

helper iterates the allowlist and invokes `Schema.SObjectField.getDescribe().isCreateable() / isUpdateable() / isAccessible()` for each field; any field that the running user lacks FLS on causes a `DocGenException` before the SOQL/DML executes. This implements the AND half of the AppExchange v1.56 review’s stated finding-resolution language (“enforce CRUD checks on the object **AND** FLS checks on the fields”). Total: **243 guard call sites across 19 controllers** (70 DML guards in 9 controllers + 173 SOQL guards in 18 classes). The guards run in addition to the v2.0 object-level `Schema.sObjectType.<Object>.isAccessible|isCreateable|isUpdateable()` gates at every admin and guest `@AuraEnabled / @InvocableMethod` entry point. `SYSTEM_MODE` is retained on the actual SOQL/DML for the documented FLS-propagation reason (`USER_MODE` strips namespaced custom fields when permset-granted FLS hasn’t propagated within the test transaction — see `DocGen_False_Positive_Report.md §2`).

7.1 Flow Invocable Actions (v1.42+)

`DocGenSignatureFlowAction` is an `@InvocableMethod` entry point that lets admins automate the full signature request lifecycle from Flow:

- **Inputs:** `templateId`, `relatedRecordId`, `signerNames[]`, `signerEmails[]`, `signerRoles[]` (optional), `signerContactIds[]` (optional), `sendEmails` (optional, **defaults to false from Flow**).
- **Outputs:** `signatureRequestId`, `signerUrls[]` (one signing URL per signer, in input order), `signerNames[]`, `signerEmails[]`, `signerRoles[]`.
- **Security:** Runs with the authenticated Flow user’s sharing (with sharing). Validates all inputs (null/blank/length-mismatch) via `DocGenException` before touching the database. Delegates actual record creation to `DocGenSignatureSenderController.createTemplateSignerRequestWithOrder()` — the same entry point the LWC signature sender uses, so both paths share identical permission-set gating and token generation.
- **Notification model:** The invocable defaults to **silent** (no package-sent emails) so the Flow author owns the notification path — typical use is Send Email Action with a customized template, or posting to Slack/Teams/Chatter via an HTTP-callout invocable. Setting `Send Branded Emails = true` uses the package’s built-in branded invitation emails instead. The LWC signature sender path is unchanged and still sends branded emails by default.

`DocGenGiantQueryFlowAction` is a sibling `@InvocableMethod` that exposes the giant-query async bulk-render pipeline to Flow for the >2000-child-row case — same Flow-user sharing model, same input-validation discipline, and it dispatches into the same batch queueables the UI bulk-render uses.

8. Basic Usage Instructions

8.1 Initial Setup

1. Install the managed package.
2. Assign the DocGen Admin permission set to admin users.
3. Assign the DocGen User permission set to end users.

4. Enable the “**Use the Visualforce PDF Rendering Service for Blob.toPdf() Invocations**” Release Update (Setup → Release Updates). This is required.
5. Add the docGenRunner LWC component to record page layouts that should offer document generation.

8.2 Creating a Template

1. Open the DocGen app (App Launcher → DocGen).
2. Click **Create New** in the template library.
3. Name the template and select the base object (e.g., Account).
4. Build the query with the visual query builder, or switch to the manual SOQL editor for legacy configs.
5. Upload a .docx file containing merge tags such as {Name} or {#Contacts}{FirstName}{/Contacts}.

8.3 Generating a Document

1. Open any record page that has the DocGen component.
2. Pick a template.
3. Choose **Download** or **Save to Record**.
4. Click **Generate Document**.

8.4 E-Signatures

1. Create a Salesforce Site that points to DocGenSignature.page.
2. Assign the DocGen Guest Signature permission set to the site guest user.
3. Set email deliverability to **All Email**.
4. Configure branding and (optionally) an Org-Wide Email Address in Command Hub → Signatures tab.
5. Add {@Signature_<RoleName>} placeholders to your Word template.
6. From a record page, use the Signature Sender component to create a signature request.

8.5 Document Verification

1. Every signed PDF includes an Electronic Signature Certificate.
2. The certificate contains a verification URL pointing at DocGenVerify.page.
3. Anyone (no login required) can verify by visiting the URL and selecting the PDF file.
4. The browser computes a SHA-256 hash locally — the PDF is **never** uploaded — and compares it against the audit record’s stored hash.

9. Security Compliance

Check	Status
Salesforce Code Analyzer	0 Critical / 0 High / 0 Moderate / 0 Low / 0 Info [†]

Check	Status
(Security + AppExchange)	
External callouts	None
Session ID usage	None
Data exfiltration paths	None identified
SOQL injection	Schema allowlist validation on all dynamic identifiers; literal binds only
XSS	Auto-escaping in LWC / Visualforce; HTML-escape before <code>Blob.toPdf()</code> input; token format validation
CSRF	Platform-handled for all <code>@AuraEnabled</code> methods
CRUD / FLS	<code>USER_MODE</code> on user-facing SOQL; object-level Schema-CRUD gate (v2.0) + per-field <code>DocGenFlsGuard</code> describe checks (v2.1.0 — 243 call sites) on package-internal SOQL/DML
Sharing model	with sharing on admin entry points; without sharing only on token-gated guest paths
Apex tests	1,449 local tests, 76% org-wide coverage
End-to-end test suite	11 chained anonymous Apex scripts (<code>scripts/e2e-01</code> through <code>e2e-08</code> + four <code>e2e-07-syntax*</code> variants) run on every release

[†] 0/0/0 against the v2.1.0 source tree achieved by two mechanisms: (1) **562 findings genuinely closed** (FLS Create 118 + FLS Update 104 + `USER_MODE` Missing 340) via `DocGenFlsGuard` real runtime enforcement; (2) **38 documented false positives suppressed at rule level** in `code-analyzer.yml` (29 `pmd:ProtectSensitiveData` field-name pattern matches on signature/audit/branding fields whose protection is enforced structurally + 9 `pmd:AvoidLwcBubblesComposedTrue` on the recursive `docGenTreeNode` LWC where `composed: true` is structurally required to cross shadow DOM boundaries). Full structural justification documented in the YAML and in `DocGen_Code_Analyzer_Report.md`. Checkmarx CxSAST will continue to flag these patterns because it doesn't trace into the `DocGenFlsGuard` helper; the rebuttal in `DocGen_False_Positive_Report.md` points at the call sites.

Supplementary documentation:

- `docs/code-analysis/violations.md` — Code Analyzer run output.
- `docs/code-analysis/checkmarx-findings.md` — Checkmarx review notes.
- `SECURITY.md` — vulnerability disclosure policy and high-level security design.
- `CLAUDE.md` — engineering invariants that protect the heap and image pipelines.
- `SECURITY_REVIEW_RESPONSE_v2.md` (this folder) — per-finding map for this re-submission with commit references.
- `DocGen_False_Positive_Report.md` (this folder) — v2.1.0 `DocGenFlsGuard` per-field rationalizations for the Checkmarx false-positive disposition.
- `DocGen_Code_Analyzer_Report.md` (this folder) — Salesforce Code Analyzer run results for v2.0.

- DocGen_Platform_Technology.md (this folder) — Salesforce platform tech inventory.

Portwood Global Solutions — <https://portwood.dev>